

TASK 2: Milestone-2

In **Milestone 1**, you built the structural outer shell: database tables, user registration, and data ingestion endpoints (where a user saves their age, sleep hours, and water intake).

In **Milestone 2**, you are building the **brain** of the application. You take that raw data sitting in PostgreSQL and MongoDB and pass it through algorithm pipelines to evaluate the user's skin and map out a strict calendar-based routine.

During these two weeks, you will focus on developing the following workflows and engines:

- **Implement the Skin Assessment Engine:** Build the logic to evaluate skin health, identify explicit concerns, and prioritize them.
- **Build Skin Concern Analysis Workflows:** Create systems to analyze specific issues like acne, hyperpigmentation, dark spots, wrinkles, or oiliness.
- **Develop Personalized Skincare Routine Generation:** Program the logic to generate tailored morning, evening, weekly, and even seasonal skincare routines.
- **Create Skin Scoring Workflows:** Build the mathematical backend for the Weighted Scoring Model. Your code will calculate the overall Skin Health Score based on:
 1. Skin Condition Assessment (35%)
 2. Lifestyle Habits (20%)
 3. Routine Consistency (20%)
 4. Sleep Quality (15%)
 5. Hydration Level (10%)
- **Generate Skincare Recommendations:** Connect the assessment data to output actionable insights and personalized guidance for the user.

Milestone 2 is the core analytical phase of the project. Without it, your application is just a static database that stores user profile details like age, sleep hours, and water intake.

You **must** do Milestone 2 to:

- **Activate the AI Engine:** It transitions your platform from simple data storage (Milestone 1) into an interactive, intelligent system that evaluates data.
- **Deliver the Primary Value Proposition:** The core objective of this entire project is to provide a *Personalized Skincare Planner*. Milestone 2 builds the specific mathematical systems and generation routines required to fulfill that promise.
- **Establish a Metric-Driven Baseline:** By calculating the automated Skin Health Score now, you create a baseline that allows users to track their progress over time in Milestone 3.

Tools to Use in Milestone 2:

According to the project's technical stack, you will use the following specific tools to build out the Milestone 2 backend, frontend, and data features:

Backend & AI Engineering

- **Python & FastAPI:** To write the business logic, calculate the scoring model, and expose REST endpoints (like `/api/v1/assessment` and `/api/v1/routine`).
- **Scikit-learn / Custom Rule Algorithms:** To build the predictive classifications for skin concern severity and type mapping.
- **Pandas & NumPy:** To parse user metrics, manage matrix lookups, and compute the floating-point mathematical equations for the weighted scoring model.

Frontend & UI

- **React.js & JavaScript:** To build the questionnaire component, the interactive routine checklist interface, and connect the UI to the backend APIs.
- **Tailwind CSS:** To design a clean, scannable, and modern user dashboard.

Data Layer

- **PostgreSQL:** To record historical skin assessment logs, active user scores, and structured step-by-step routine plans.
- **MongoDB:** To track dynamic daily checklist submissions and user lifestyle log details.

Step 1: Database Setup & Matrix Seeding:

Goal: Prepare your data layer to store generated routines, maintain score histories, and hold the basic rules mapping skin types to routine structures.

1.1 Create Database Migrations (PostgreSQL)

Define and execute the SQL schemas for your relational structures:

- **skin_assessments Table:**
 - **id** (UUID, Primary Key)
 - **user_id** (UUID, Foreign Key linking to Users table from Milestone 1)
 - **overall_score** (Numeric / Float)
 - **detected_concerns** (JSONB array: e.g., ["Acne", "Dry Skin"])
 - **created_at** (Timestamp)
- **skincare_routines Table:**
 - **id** (UUID, Primary Key)
 - **user_id** (UUID, Foreign Key)
 - **assessment_id** (UUID, Foreign Key linking to **skin_assessments**)
 - **time_of_day** (VARCHAR: AM, PM, or Weekly)
 - **step_number** (Integer)
 - **step_category** (VARCHAR: e.g., Cleansing, Treatment, Moisturizing, Sun Protection)
 - **is_active** (Boolean, Default: True)

1.2 Create Database Schema (MongoDB)

Define the unstructured document structure for daily tracking:

- **routine_logs Collection:**
- JSON

```
{  
  
  "user_id": "string_or_uuid",  
  
  "log_date": "YYYY-MM-DD",  
  
  "completed_steps": [  
  
    { "routine_step_id": "string", "completed_at": "timestamp" }  
  
  ],  
  
  "water_intake_ml": 2500,  
  
  "sleep_hours": 7.5  
  
}
```

1.3 Seed the Decision-Matrix Matrix

Because product catalog scraping happens in Milestone 3, seed a temporary internal lookup schema (or table) mapping specific user skin types to standard skincare steps:

- Mapping Entry A: If Skin Type is **Oily** $\rightarrow$ AM Steps: [Cleansing, Treatment, Sun Protection], PM Steps: [Cleansing, Treatment, Night Care].
- Mapping Entry B: If Skin Type is **Sensitive** $\rightarrow$ AM Steps: [Cleansing, Moisturizing, Sun Protection], PM Steps: [Cleansing, Moisturizing].

Step 2: Build the Skin Assessment Engine (Backend)

Goal: Extract user inputs (from Milestone 1) and algorithmically determine their prioritized skin problems and condition categories.

2.1 Set Up the FastAPI Endpoint

Create a route in your FastAPI app: `POST /api/v1/assessment/evaluate` (Ensure it is protected by the JWT authentication layer built in Milestone 1).

2.2 Write the Concern Prioritization Code

Inside your service layer, build an extraction algorithm using **scikit-learn** or a deterministic rule-based framework:

- Write a function `identify_skin_concerns(profile_data)` that scans the user's self-reported issues (e.g., fine lines, acne flare-ups, redness).
- Sort and flag these concerns by severity using conditional matrices or logic weight maps (e.g., if a user selects both "Acne" and "Wrinkles," but ranks acne as an active flare-up, prioritize **Acne** as the primary concern).

Step 3: Code the Weighted Scoring Logic (Backend)

Goal: Write the mathematical calculation engine that evaluates skin health and updates the system score.

3.1 Implement the Mathematical Function

Core Objective: The document outlines how to build the mathematical backend (Step 3) that calculates a comprehensive **Skin Health Score** out of 100, saves it to a database, and delivers it to a front-end application.

In a file named `scoring_engine.py`, write a function utilizing **NumPy** or standard math libraries to execute the exact weighted equation.

The Scoring Equation: The system calculates a single overarching score using five specific weights:

$$\text{Skin Health Score} = (S_{\text{cond}} \times 0.35) + (L_{\text{habits}} \times 0.20) + (S_{\text{sleep}} \times 0.15) + (R_{\text{consist}} \times 0.20) + (H_{\text{hydro}} \times 0.10)$$

- **Calculate S_{cond} (Skin Condition - 35%):** Start at 100. Subtract 15 points for every "High Severity" concern identified in Step 2, and 7 points for "Medium Severity" concerns.
- **Calculate L_{habits} (Lifestyle - 20%):** Start at 100. Deduct points if environmental exposure metrics show high unprotected UV index exposure.
- **Calculate S_{sleep} (Sleep - 15%):** Evaluate recorded sleep data against the ideal 8-hour target ($(\text{Hours Logged} / 8) \times 100$, capped at 100).
- **Calculate R_{consist} (Consistency - 20%):** Fetch the last 7 days of routine logs from MongoDB. Calculate the percentage of checks completed.
- **Calculate H_{hydro} (Hydration - 10%):** Compare recorded water intake metrics against standard recommendations.

3.2 Create the Output Route

Expose this algorithm via `GET /api/v1/assessment/score` so the front-end can request a user's latest score break-down. Save the results to your `skin_assessments` table.

1. Calculate and Return the Skin Score Breakdown

When the front-end application sends a request to this route, your backend will execute the scoring algorithm outlined just above it in the document. It calculates a personalized skin health score based on five weighted metrics:

- **Skin Condition (S_{cond} - 35%):** Starts at 100, deducting points for high (15 pts) or medium (7 pts) severity skin concerns.
- **Lifestyle (L_{habits} - 20%):** Deducts points for high unprotected UV index exposure.
- **Sleep (S_{sleep} - 15%):** Evaluates logged sleep data against an ideal 8-hour target.
- **Consistency (R_{consist} - 20%):** Calculates the completion percentage of skincare routines over the last 7 days.
- **Hydration (H_{hydro} - 10%):** Compares water intake against standard recommendations.

The front-end will use this returned data to display a comprehensive, broken-down score dashboard to the user.

2. Persist the Data (Save History):

Before (or at the same time) the API sends the response back to the front-end, it must **write a new record into your database** inside the `skin_assessments` table.

This ensures that every time a score is calculated, a historical snapshot is saved. This allows the app to track the user's skin journey over time, power historical charts, and keep a log of how their lifestyle and skin conditions change.

Step 4: Write the Routine Generation Logic (Backend)

Goal: Map out a specific calendar workflow for the user based on their newly generated health metrics.

4.1 Write the Step Mapping Function

Create the route: `POST /api/v1/routine/generate`.

- Have the function ingest the prioritized primary concern and baseline skin type output from Step 2.
- Query your seed data from Step 1.3 to pull the corresponding template array.
- If the user's skin is flagged as highly sensitive, add a safety check condition in your Python logic to intercept the array and drop or swap harsh categories (like replacing aggressive physical exfoliation steps with gentle moisture treatments).
- Save the generated array of objects step-by-step into the `skincare_routines` table assigned to that specific `user_id`.

Step 5: Build Frontend Views & Core Connections (React.js)

Goal: Expose the backend logic through an interactive user dashboard built with React and Tailwind CSS.

5.1 Build the Multi-Step Assessment Form

Objective: Create a multi-step form (wizard) in React to collect user data (skin concerns, sleep, hydration, and lifestyle). When completed, send this data to our backend API and redirect the user to their dashboard.

1. Multi-Step Form UI

- Create a single parent component that uses a step counter state: `const [step, setStep] = useState(1);`.
- Conditionally render one sub-form per step (e.g., Step 1: Skin Concerns, Step 2: Habits).
- Add a visual progress bar at the top showing the current step (e.g., "Step 2 of 4").
- **Data Saving:** Save the form inputs to `localStorage` on every change. If the user refreshes, reload their progress so they don't have to start over.

2. Input Validation

- Do not let the user click "Next" if mandatory fields on the current step are empty or invalid.
- Use a validation library (**Zod** or **Yup**) to enforce rules (e.g., sleep hours must be a number between 0 and 24).

3. Submit Data & Handle Loading

- On the final step, use **Axios** or **Fetch** to send the final JSON data object via a `POST` request to: `/api/v1/assessment/evaluate`.
- **Loading State:** While waiting for the API response, disable all buttons and show a loading spinner with the text: "Analyzing your skin profile..."

4. Error Handling & Success Routing

- Wrap your API call in a `try/catch` block. If the server returns an error (e.g., network failure), show a red error banner at the top of the form instead of letting the app crash.
- On a successful response (`200 OK`), clear the draft data from `localStorage` and use `react-router-dom` to automatically send the user to the `/dashboard` route.

5.2 Build the Daily Planner Dashboard & Checklist

Objective: Create the main user dashboard in React. This dashboard will fetch the user's customized skincare routine from the backend, display it in clean grids, and allow them to check off tasks in real-time.

1. Dashboard Layout & Grid System:

Design a responsive layout split into three distinct visual grid cards:

- **Morning Plan (AM)**
- **Evening Plan (PM)**
- **Weekly Highlights**

2. Fetch Routine Data:

- Use a React `useEffect` hook to automatically make a `GET` request to `/api/v1/routine` when the dashboard loads.
- Save the response array into your local component state and map through it to dynamically render the routine steps inside the **AM** and **PM** grids.

3. Interactive Checkboxes:

- Add a standard interactive checkbox next to each rendered routine step (e.g., `[ ] Step 1: Cleansing`).
- The checkbox should reflect the completion status fetched from the backend.

4. Compliance Tracking (State Updates):

- Attach an `onChange` event handler to each checkbox.
- When a user clicks a checkbox, immediately fire a `POST` request to the backend. This request must update the MongoDB `routine_logs` collection to permanently save whether they completed or uncompleted that specific step.

Step 6: End-to-End Validation & Unit Testing

Goal: Write automated unit tests in Python to verify that our backend scoring engine and routine generator handle calculations and safety logic correctly before we wrap up this milestone.

6.1 Run Automated Unit Tests (Python/FastAPI)

Create unit tests in your testing folder to validate isolated code blocks.

Test 1: Math & Score Precision:

- **Goal:** Ensure a perfect data input outputs a perfect score.
- **Action:** Write a test function that inputs ideal metrics into `calculate_skin_health_score()` (e.g., zero skin concerns, perfect 8 hours of sleep, excellent lifestyle habits).
- **Assertion:** Use an assert statement to verify that the returned final score equals exactly `100.0`.

Test 2: Routine Safety Boundaries:

- **Goal:** Ensure the system never gives harsh products to sensitive skin.
- **Action:** Write a test function that mimics a user profile with "Severe Skin Sensitivity" or severe acne barriers. Pass this mock profile into your routine generation function.
- **Assertion:** Verify that the generated skincare routine automatically excludes harsh chemical steps (like strong exfoliating acids or intense retinoids) and replaces them with safe alternatives.

6.2 Manual Integration Verification

1. Authenticate & Log In:

- Open Postman or your local frontend client and log into a test user account to obtain a valid session/token.

2. Submit an Assessment Form:

- Complete and send the skin health assessment data (simulating the Step 5.1 wizard form) via a **POST** `/api/v1/assessment/evaluate` request.

3. Verify PostgreSQL Storage:

- Open your PostgreSQL database client and query the `skin_assessments` table.
- **Check:** Confirm that a new record was successfully saved with your calculated score breakdown.

4. Verify Routine Generation

- Send a **GET** `/api/v1/routine` request.
- **Check:** Confirm that a customized array containing both **Morning (AM)** and **Evening (PM)** skincare steps returns successfully.

5. Track a Completed Step :

- Toggle a routine step checkbox (simulating the Step 5.2 dashboard) to send a update request to the backend.
- **Check:** Open your MongoDB client, check the `routine_logs` collection, and confirm that your checked task was securely logged as completed.

Delivered at the End of Milestone 2

By the end of this milestone, the following items must be fully developed, integrated, and verified:

1. Backend Engineering (Python / FastAPI)

- **The Skin Assessment Engine:** Logic to evaluate a user's skin profile, identify explicit skin concerns, and prioritize them.
- **Skin Concern Analysis Workflows:** Workflows that can specifically analyze target issues like acne, hyperpigmentation, dark spots, wrinkles, or oiliness.
- **Weighted Scoring Model:** A core script (`scoring_engine.py`) that executes the exact mathematical equation to generate a single Skin Health Score out of 100 based on skin condition (\$35\%\$), lifestyle (\$20\%\$), sleep (\$15\%\$), consistency (\$20\%\$), and hydration (\$10\%\$).
- **The Score API Route:** A working `GET /api/v1/assessment/score` endpoint that runs the scoring engine, returns the breakdown to the frontend, and logs a historical snapshot into the PostgreSQL `skin_assessments` table.
- **Personalized Skincare Routine Generation:** Algorithm logic that automatically generates a tailored morning (AM), evening (PM), weekly, and seasonal skincare routine plan based on the user's profile.

2. Frontend Engineering (React)

- **Multi-Step Assessment Form Wizard:** A user-friendly, step-by-step questionnaire that validates user input on every step, caches progress in `localStorage`, and sends the final profile payload to `POST /api/v1/assessment/evaluate` before routing the user to their dashboard.
- **Daily Planner Dashboard & Checklist:** A three-card grid layout displaying the **Morning Plan (AM)**, **Evening Plan (PM)**, and **Weekly Highlights**. It must use a `useEffect` hook to fetch routine data from `GET /api/v1/routine`.
- **Live Compliance Checkboxes:** Interactive checkboxes next to each routine step that trigger an `onChange POST` request to instantly save completion records in the MongoDB `routine_logs` collection.

3. Quality Assurance & Validation

- **Automated Unit Tests:** A Python test suite with tests verifying math precision (ideal inputs yield a perfect `100.0` score) and safety boundaries (sensitive skin profiles automatically exclude harsh chemicals).
- **End-to-End Integration Verification:** A verified manual test run (via Postman or the frontend client) proving a user can successfully log in, submit an assessment, save data to PostgreSQL, fetch a generated routine, and check off a task to write a log into MongoDB.

ARCHITECTURE DIAGRAM :

